

## Assignment 4: Code Review

### **Code smell 1: Duplicated image loading code in multiple classes**

Refactoring:

A utility class, ImageLoader, was created with a static method loadImage(String path, int width, int height) to handle image loading and scaling. This reduced redundant code in classes like Apple, Bone, Chocolate, etc, where first sprites were loaded and then they were scaled. This function instead manages to do it all in once, improving the consistency of the code.

Used in: Apple, Exit.java, Whiskey.java, Steak.java, Mushroom.java, Bone.java and Chocolate.java

Commit: 8e9e2bb4acdf0eb43af5c725583a2f0aaf3ddb0

### **Code smell 2: Multiple getBounds Method Across Classes**

Refactoring:

The getBounds method was moved to the abstract class GameObject, which is extended by other classes. This reduced duplication and centralized the logic for bounding box creation. A width and height variable were added to the GameObject to make the method flexible across different subclasses

Used in: Apple.java, Bone.java, Mushroom.java, Bush.java, Chocolate.java, Exit.java, Onion.java, Rat.java, Steak.java, Whiskey.java

Commit: c4127ebd3205b81eed665947e2997ae70250cec2

### **Code smell 3: Feature envy - Exit Appearance Logic in Doug**

The logic for the appearance of the exit was moved from Doug to the Exit class, ensuring that the behavior is encapsulated within the class it belongs to. Exit collision previously based on calculating the score and seeing if it is larger than the total needed score which should be done in exit. By relocating this responsibility, the code achieves better cohesion.

Commit: dcf0a36ee2bf12d6d5c27af5da47ec9627dc2875

previous:

```
private void handleExitCollision(Exit exit) {
    if (Score.boneScore >= Score.boneTotal) { // Check if all bones are collected
        Game gameInstance = handler.getGameInstance(); // Access the Game instance
        if (gameInstance != null) {
            gameInstance.setGameWon(gameWon:true); // Mark the game as won
        }
    }
}
```

Refactored:

```
private void handleExitCollision(Exit exit) {
    if (!exit.getHiddenStatus()) { // Check if all bones are collected
        Game gameInstance = handler.getGameInstance(); // Access the Game instance
        if (gameInstance != null) {
            gameInstance.setGameWon(gameWon:true); // Mark the game as won
        }
    }
}
```

the if condition is replaced with a function to get hidden status

Changes in: Doug.java and Exit.java

### **Code smell 4: Removed Unused Methods and comments**

Refactoring:

Unused methods, such as removed Listener in GameOverScreen and getTickCount in Handler, were removed, in addition comments that were written in the code as a guide for ourselves (not javadocs) were deleted since methods were already implemented. This cleanup reduces code clutter and improves readability, making the code only focus on the functional and necessary methods.

Commit: 63471f0305cfa0d82c970c7b64e004f5647a500f

### Code smell 5: Long List of parameters - Making handler a singleton

Refactoring

By making the Handler class a singleton, the need to pass the handler as a parameter to multiple objects has been eliminated, addressing the code smell of a long list of method parameters. This change simplifies method calls, reduces redundancy and makes the handler globally accessible while maintaining its control over its single instance.

handler.java

Commit: 94d7297c9ad3c6c7857932d8830cebf78039764d

```
/**
 * Returns the main Handler instance associated with this handler.
 *
 * @return The main Handler instance.
 */
public static synchronized Handler getHandlerInstance(Game game){
    if (handler == null) {
        handler = new Handler(game);
    }
    return handler;
}
```

### Code smell 6: Duplicated code

making enemy movement based in the Moving Enemy class so that future moving enemies could be implemented - duplicate code

Rat class held the entirety of the enemy movement making it so that if we chose to make another moving enemy class it would be necessary to copy and paste the movement from tick into another moving enemy. Moving the functionality from the rat to the moving enemy class allows for different enemies to share the same movement mechanic without copy and pasting code

Commit: 91a925a31ff704a1d684edb21ae6a6f18b45374d

MovingEnemy.java

Rat.java

```
protected void move() {
    Doug doug = Doug.getInstance();

    if (doug != null) {
        // Target position (Doug's position)
        int targetX = doug.getX();
        int targetY = doug.getY();

        // Determine preferred direction
        int preferredVelX = (x < targetX) ? 1 : (x > targetX) ? -1 : 0; // Move horizontally
        int preferredVelY = (y < targetY) ? 1 : (y > targetY) ? -1 : 0; // Move vertically

        // Try moving along horizontal axis first
        if (!tryMove(preferredVelX, dirY:0)) {
            // If horizontal is blocked, try moving vertically
            if (!tryMove(dirX:0, preferredVelY)) {
                // If both are blocked, stop moving
                velX = 0;
                velY = 0;
            }
        }
    }
}
```

```
/**
 * What the object should do on each tick
 */
@Override
public void tick() {
    move();
}
```

### Code smell 7: Badly structured project

Refactoring:

Files were reorganized to follow a logical structure, grouping related classes (e.g., GameObjects, GameProperties, Rewards, Trackers, etc.) into appropriate packages. This not only improves navigation within the project, but also the project modularity in general.

Commit: 85b5845551f9f70acb43ef812d1db658c3ec9748

### Code smell 8: Inconsistent exception handling

Refactoring:

Consistent and informative exception handling was implemented across the code. Each catch block now includes a descriptive error message that is printed in the console, explaining the nature of the exception and its content. This ensures better debugging support and improves overall quality for exception handling.

Commit: 57c99bf6a0709fd767a0418203310390a18a4395